beonic

API - Venues and floors V5

Beonic Support
1 year ago · Updated

Follow

Architecture	REST
Protocol	HTTPS
Method	GET
Request Format	JSON
Endpoint	Regional
Security	TLS, Basic Authentication
Scope	Venues

Create your API Key

Follow this article to Create your API Key in the Venues Scope - Create API key

Refs

Generally across the venues API, entities can be referred to via name or id. The format is:

```
{"name": "Some name"}
```

or

```
{"id": 12345 }
```

A ***ref*** in the documentation below refers to one of the above formats. If both name and id are specified in a ref, the id takes precedence.

Refs can also be specified as query params:

```
/v5/api/venues?name=Name
```

or as path fragments:

```
/v5/api/venues/name:my_venue/floors/name:Only Floor
```

In path fragments, a number alone is the same as id. So the following are equivalent:

```
/v5/api/venues/123/floors
```

and

```
/v5/api/venues/id:123/floors
```

Create Venue

```
POST /v5/api/venues
```

Request body

- name (required) - the name of the new venue
- timezone (required) - the venue's time zone
- enabled (optional) - a boolean to indicate whether the venue is enabled
- registrationGroup (required) - a ref to the venue's registration group.
- marketingList (required) - a ref to the venue's marketing list
- floors - an array of floors for the new venue Floor objects with the format:
  - `{"name": "Floor name"}`
- userAccessGroups: an array of user access groups, or refs to existing user access groups. User access groups have one of the following formats:
  - `{"name": "my group", "description": "some group description"}`
  - `{"ref": ref}`
-

Response body

- name - the venue name
- timezone - the venue timezone
- registrationGroup - the registration group. an object containing id and name fields
- enabled - boolean stating whether this venue is enabled
- userAccessGroups - an array of this venue's user access groups. array members are objects containing id and name fields
- marketingList - the marketing list . an object containing id and name fields
- id - the venue's id

- floors - an array of floors - each floor contains the following fields:
  - id - floor id
  - name - floor name
  - zones - array of zones. each zone contains an id and a name
  - smartZones - array of smart zones. each contains id and name

Sample

```
POST /v5/api/venues

Request:

{
  "name": "Today venue 12",
  "timezone": "UTC",
  "registrationGroup": {"name": "Office"},
  "marketingList": {"id": 691},
  "floors": [{"name": "first floor"}],
  "userAccessGroups": [{"ref": {"name": "Office"}}]
}

Response:

{
  "name": "Today venue 12",
  "registrationGroup": {
    "id": 595,
    "name": "Office"
  },
  "timezone": "UTC",
  "enabled": true,
  "userAccessGroups": [
    {
      "id": 964,
      "name": "Office"
    }
  ],
  "marketingList": {
    "id": 691,
    "name": "Office"
  },
  "id": 8927,
```

```
  "floors": [
  {
    "id": 13367,
    "name": "first floor",
    "zones": [],
    "smartZones": []
    }
  ]
}
```

Update Venue

```
PUT /v5/api/venues
```

Request body:

- id (either this or ref are required) - venue id
- ref (either this or id are required) - a **ref** referring to the venue
- name (optional) - the new name of the venue
- timezone (optional) - the venue's time zone
- enabled (optional) - a boolean to indicate whether the venue is enabled
- registrationGroup (optional) - a ref to the venue's registration group.
- marketingList (optional) - a ref to the venue's marketing list
- floors (optional) - an array of floors, containing either new floors or refs to existing floors. Formats are :
  - `{"name": "Floor name"} // Creates new floor`
  - `{"ref": ref} // References existing floor`
  - `{"ref": ref, "name": "Floor name 2"} // References and updates ex`
  - `{"id": 123, "name": "Floor name 2"} // Shorthand for "ref": {"id"`

- userAccessGroups: an array of user access groups, or refs to existing user access groups. User access groups have one of the following formats:
  - `{"name": "my group", "description": "some group description"}`
  - `{"ref": ref}`

Response body:

The response is the same as above

Sample

```
PUT /v5/api/venues

Request:

{
  "name": "Demo venue 2",
  "timezone": "UTC",
  "enabled": true,
  "ref": {"id": 12782},
  "floors": [
    {
      "id": 19996,
      "name": "ground floor",
     },
     {
         "ref": {"name": "second floor"}
      }
    ]
}

Response:

{
  "name": "Demo venue 2",
  "registrationGroup": {
    "id": 595,
```

```
    "name": "Office"
  },
  "timezone": "UTC",
  "enabled": true,
  "userAccessGroups": [
    {
      "id": 964,
      "name": "Office"
    }
  ],
  "marketingList": {
    "id": 691,
    "name": "Office"
  },
  "id": 8928,
  "floors": [
    {
      "id": 13369,
      "name": "ground floor",
      "zones": [],
      "smartZones": []
    },
    {
      "id": 13370,
      "name": "second floor",
      "zones": [],
      "smartZones": []
    }
   ]
}
```


Get venue

```
GET /v5/api/venues/ref
```

Response body:

The response is the same as above

Sample

```
GET /v4/api/venues?name=Demo venue 2

Response:

{
  "id": 13370,
  "name": "second floor",
  "zones": [],
  "smartZones": []
}
```

Get all venues

```
GET /v5/api/venues
```

Response body:

An array of venue objects, specified above.

Sample

```
GET /v4/api/venues

Response:

[{
  "id": 13370,
  "name": "second floor",
  "zones": [],
  "smartZones": []
},
...
]
```

Delete venue

```
DELETE /v5/api/venues/ref
```

Response body:

"success", or an object containing an error message

Sample

```
DELETE /v5/api/venues?name=Demo venue 2

Response:

"success"
```

List floors for venue

```
GET /v5/api/venues/ref/floors
```

Response body:

An array of floors - each floor contains the following fields:

- id - floor id
- name - floor name
- zones - array of zones. each zone contains an id and a name
- smartZones - array of smart zones. each contains id and name

Sample

```
GET /v5/api/venues/123/floors

Response:

[
{
  "id": 13369,
  "name": "ground floor",
  "zones": [],
  "smartZones": []
},
{
  "id": 13370,
  "name": "second floor",
  "zones": [],
  "smartZones": []
}
]
```

Get floor

```
GET /v5/api/venues/ref/floors/ref
```

Response body:

- id - floor id
- name - floor name
- zones - array of zones. each zone contains an id and a name
- smartZones - array of smart zones. each contains id and name

Sample

```
GET /v5/api/venues/name:Demo venue 2/floors/name:second floor
```

```
Response:

{
  "id": 13370,
  "name": "second floor",
  "zones": [],
  "smartZones": []
}
```

Delete floor

```
DELETE /v5/api/venues/ref/floors/ref
```

Response body:

"success", or an object containing an error message

Sample

```
DELETE /v5/api/venues/8928/floors/name:second floor

Response:

{}
```

Error Conditions

Errors can include format or validation errors. In the case of error, a non-200 HTTP status is returned and an object containing a message field explains what went wrong. A sample error response is below

```
{
   "message": "Venue validation failed: Venue with name 'Today venue 12' alre
}
```

Was this article helpful?

✓ Yes ✕ No

0 out of 0 found this helpful

Have more questions? Submit a request

Return to top

Recently viewed articles

API - Metrics - Counter Data

Insights API - Management screens

Entity API

API - Portal Deployment

API - Access Points

Related articles

API - Metrics - Counter Data

API - Access Points

Account API Keys

Blix Metrics API

API - Edit a User Profile

Comments

0 comments

Be the first to write a comment.

Beonic

English (GB)